CML Divide-by-3 Frequency Divider

Design and Simulation Report

Prepared by: **Kareem Ayman Saeed**

ADI Internship

CML Divider Design and Simulation

1. Objective and Design Specifications

This report documents the design and simulation of a Current Mode Logic (CML) divide-by-3 frequency divider, following the workflow described in the reference tutorial "CML Divider Design and Simulation." The divider is built from two CML D flip-flops (latches) and one CML AND gate arranged in a feedback ring, as shown in Figure 1. The target specifications are summarized below.

Parameter	Value
Supply Voltage, Vdd	1.5 V
Maximum Input Frequency, Fin	10 GHz
Division Ratio	3
Load Capacitance (single-ended)	100 fF
Differential Output Swing (peak)	300 mV
Design Goal	Minimize power consumption

The design follows the recommended approach of giving each flip-flop four differential outputs (two core CML outputs plus two source-follower-buffered outputs), using the source followers to drive the 100 fF output load capacitance without loading the regenerative core of the latch.

2. Circuit Design

2.1 Divider Top-Level Schematic

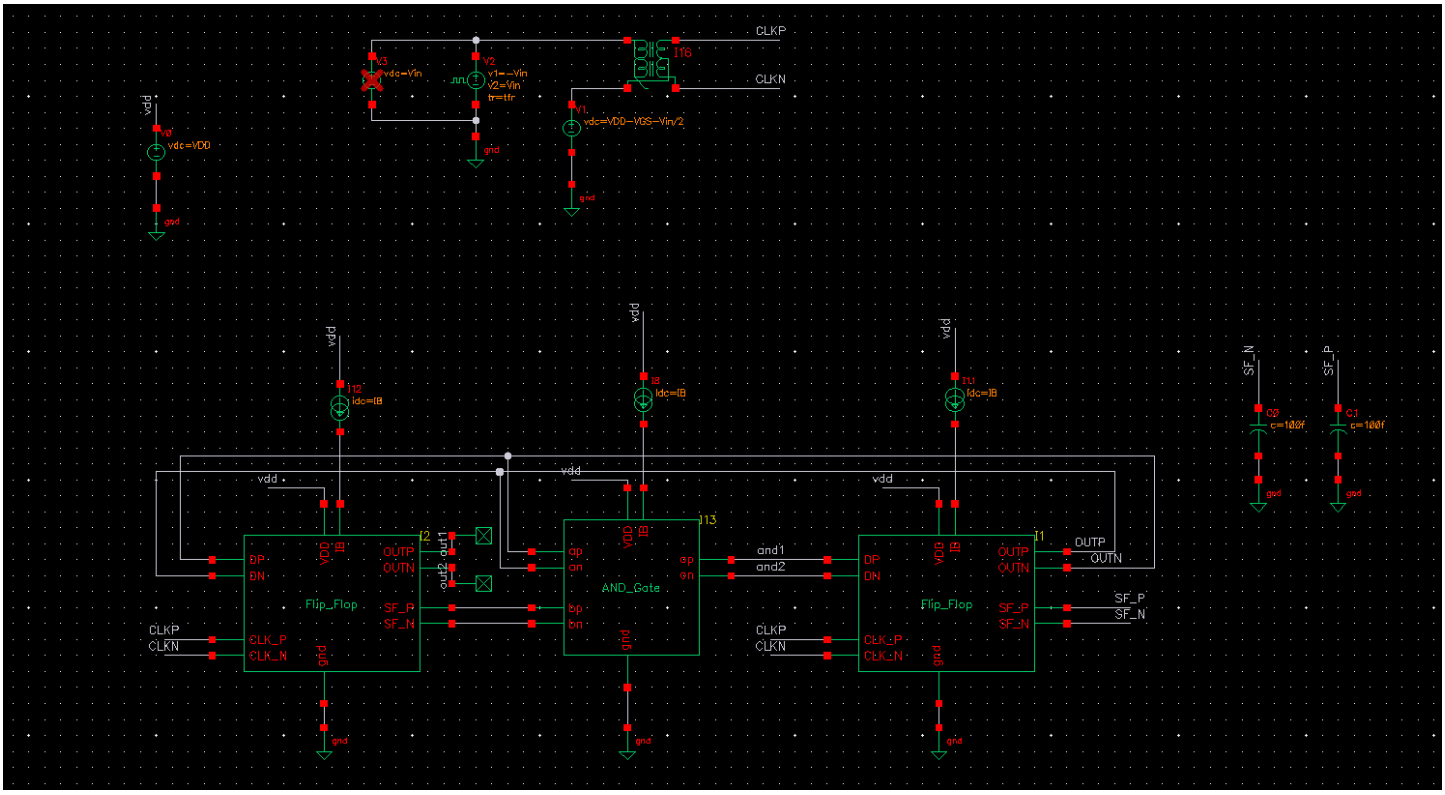

Figure 1. *Divide-by-3 top-level schematic: two CML D-latches (I1, I2) and one CML AND gate (I3) connected in a feedback ring, biased from Vdd = 1.5 V by tail current sources (idc = ibc). The differential outputs op_sf/on_sf drive the 100 fF single-ended load capacitors C0 and C1.*

Comment: The ring topology (latch → AND → latch → feedback) implements the divide-by-3 truth table using the minimum gate count, which helps meet the low-power target.

2.2 CML D Flip-Flop (Latch)

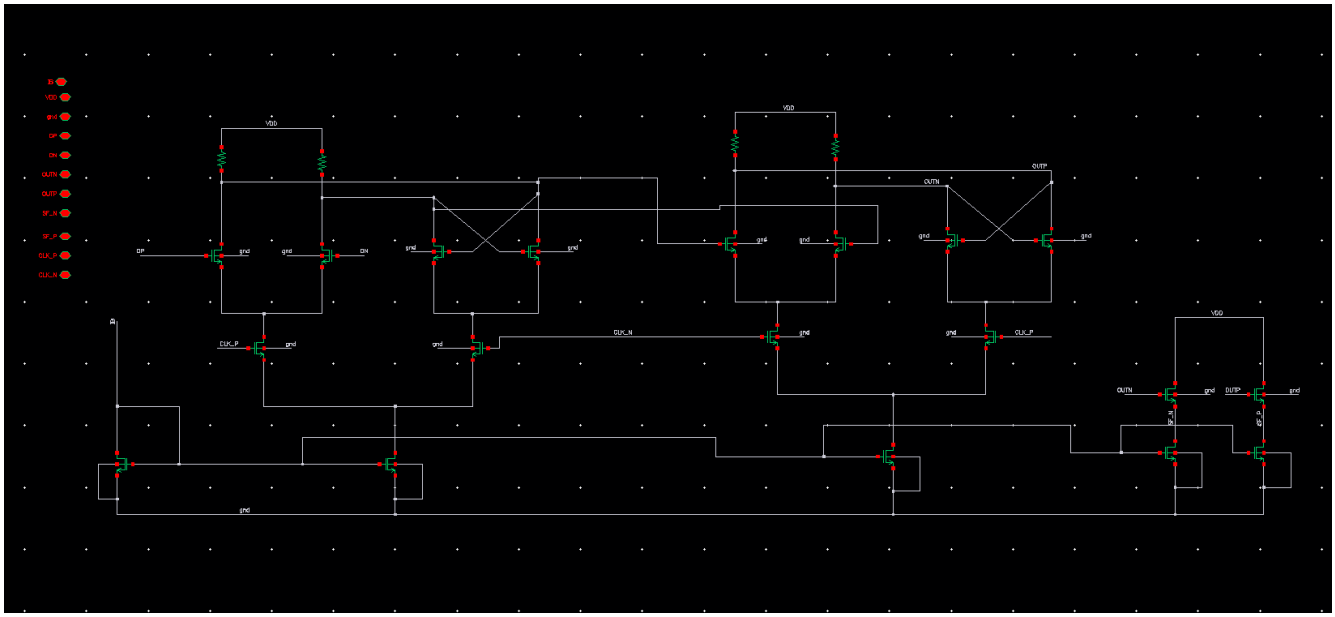

Figure 2. *CML D-latch schematic: differential input pair with cross-coupled regenerative pair, clocked by clkp/clkn, and source-follower buffers generating the load-driving outputs op_sf/on_sf.*

Comment: The core differential pair sets the regeneration (latching) speed, while the source followers isolate this fast internal node from the 100 fF external load, which is essential for meeting the 10 GHz operating speed.

2.3 CML AND Gate

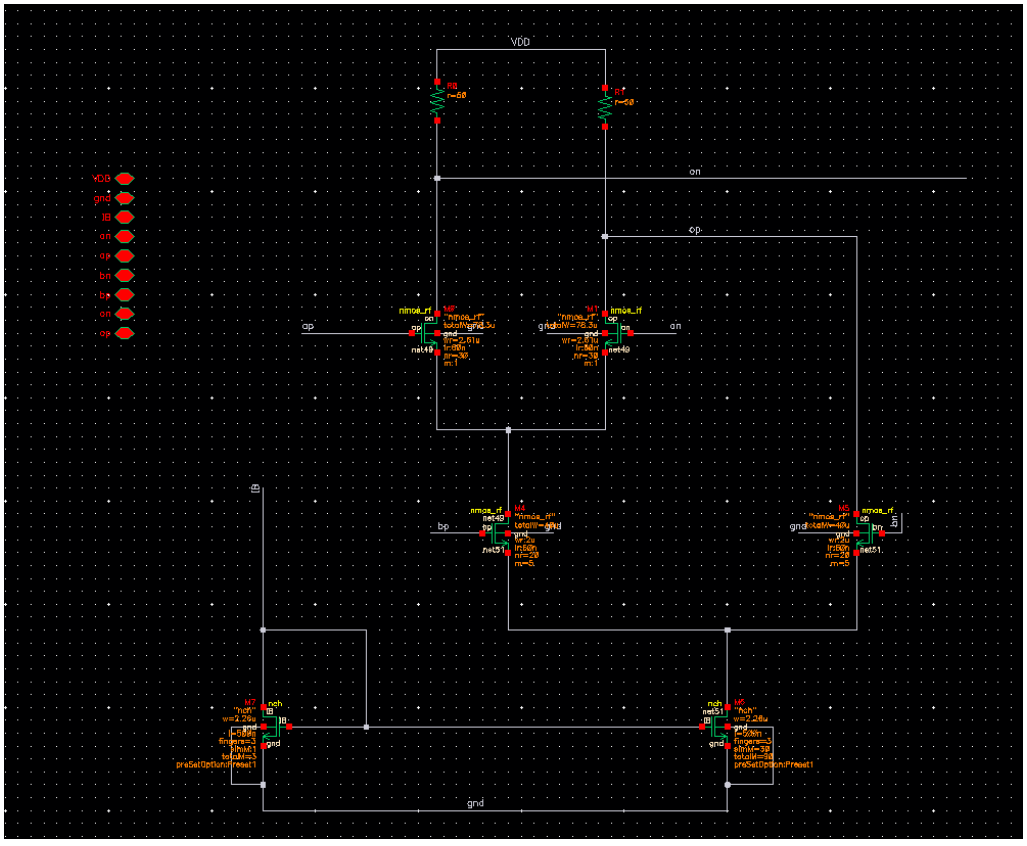

Figure 3. *CML AND gate schematic used in the feedback path to combine Q1 and CK, generating the logic term that forces division by three.*

Comment: Because the AND gate sits directly in the critical feedback loop, its delay was budgeted alongside the two latches in the Part 1 hand analysis.

2.4 Hand Delay Calculation from the DC Operating Point

Per-stage propagation delay in a CML gate is dominated by the single pole at the output node and can be estimated as:

$$\tau_{stage} \approx 0.69 \times R_L \times C_L , \quad C_L = C_{dd,self} + C_{gg,next\text{-}stage} \; (+ C_{ext} \text{ for the buffered output})$$

The load resistor is set by the schematic design parameter RL = 50 Ω (used in both the AND gate and each latch, per the r=50 parameter on the AND-gate schematic above). CL for each internal (unbuffered) node is the sum of the switching device's own drain capacitance (Cdd) and the input gate capacitance (Cgg) it drives; for the buffered outputs, CL also includes the 100 fF external load.

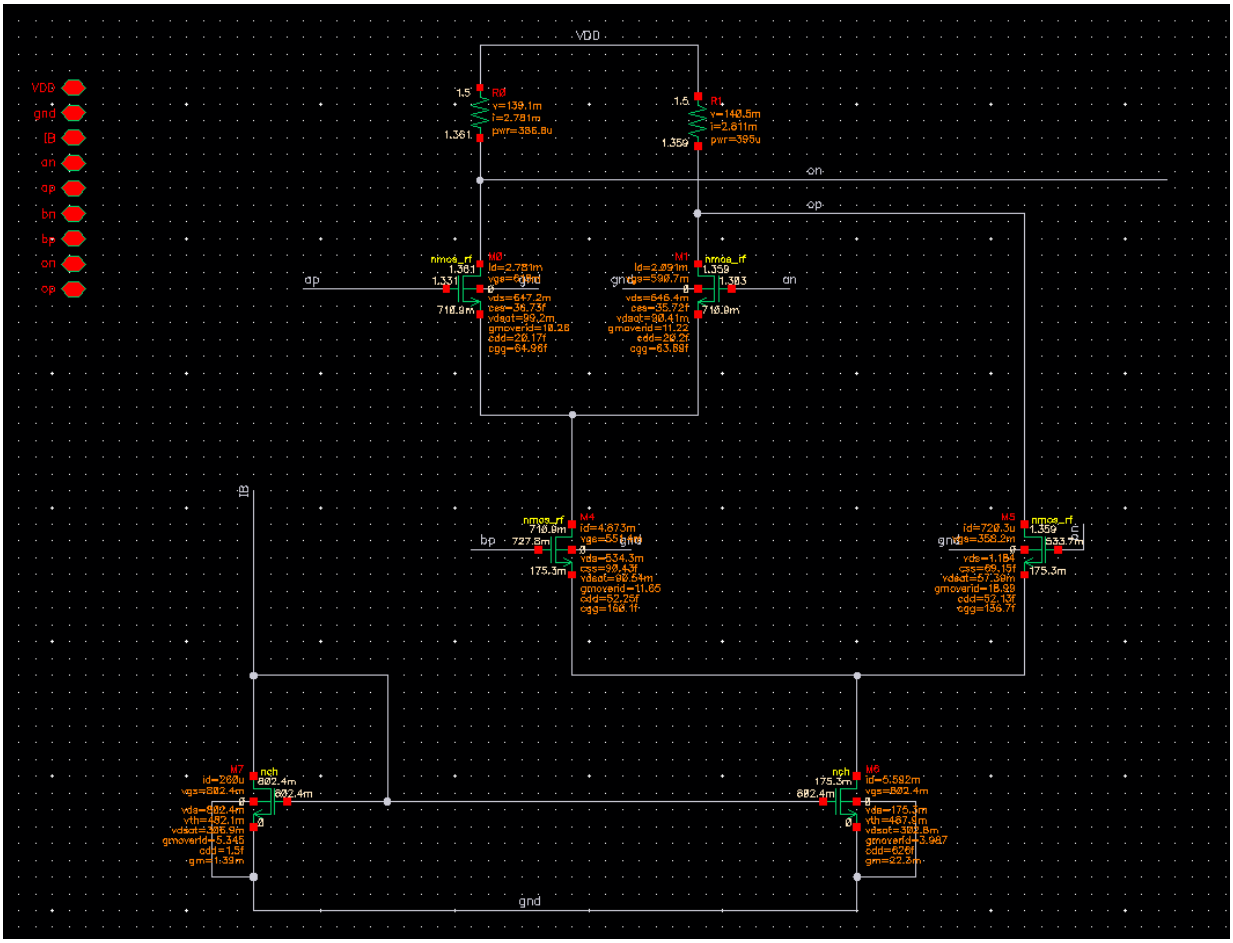

Figure 4. *DC operating point of the AND gate with both inputs biased at the input common-mode voltage, Vin,CM = Vdd − VGS − Vswing/2. The annotated Id, gm/Id, Cdd and Cgg values for M0–M6 are the inputs to the τstage estimate above.*

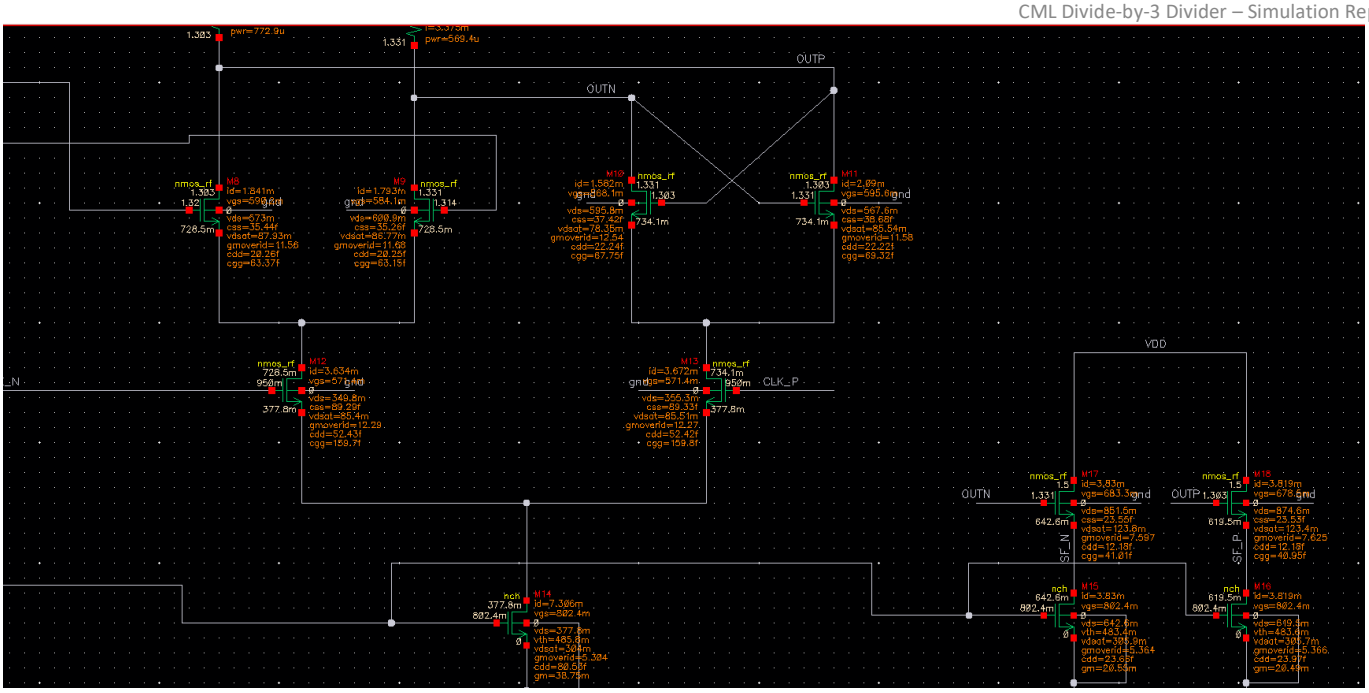

Figure 5. *DC operating point of the second D flip-flop, DFF2 (the divider-output latch), showing the same set of annotations (Id, gm/Id, Cdd, Cgg) for its regenerative pair and source-follower output stage.*

Using the DC-operating-point annotations read off Figures 5 and 6 (AND gate: M0/M1, M8/M9; DFF2: M10/M11, M17/M18), the internal (unbuffered) output node of each gate is loaded by the drain capacitance of its own switching device plus the gate capacitance of the very next stage it drives — not by the 100 fF external load, which only appears after the source follower.

Stage	CL = Cdd,self + Cgg,next (fF)	RL (Ω)	τstage = 0.69·RL·CL (ps)
AND gate (M0/M1 → M8/M9)	20.19 + 63.26 = 83.44	50	2.88
DFF2 core (M10/M11 → M17/M18)	22.23 + 40.98 = 63.21	50	2.18
DFF1 core (assumed ≈ DFF2, same topology/sizing)	≈ 63.21	50	≈ 2.18

τloop = τDFF1 + τAND + τDFF2 ≈ 2.18 + 2.88 + 2.18 ≈ 7.24 ps, only about 7% of the T = 100 ps input clock period at Fin = 10 GHz — a comfortable margin, and consistent with the Part 1 guidance to keep the digital-gate delays small relative to the clock period.

A pure RC pole, however, understates the latch: regeneration also needs enough gain×time for a small differential input to build up to full logic swing, set by τregen = CL/gm of the cross-coupled pair. Using M10 (gm/Id = 12.54 1/V, Id = 1.582 mA → gm ≈ 19.84 mS) and the same CL = 63.21 fF:

τregen = CL / gm ≈ 63.21 fF / 19.84 mS ≈ 3.19 ps ⇒ ~5–8 τregen ≈ 16–25 ps for reliable full-swing regeneration

This regeneration estimate is still well inside the 100 ps period, but it is closer to the dominant term. The remaining gap to the ~55–73 ps rise times actually measured in Section 4.5 is explained by the buffered output stage: the source followers (M17/M18, tail current ≈ 3.83 mA) charge/discharge the full 100 fF external load specified in the objective, not the small internal node used above. A simple slew estimate, t ≈ CL,ext·ΔV / Itail = 100 fF × 0.3 V / 3.83 mA ≈ 7.8 ps for a linear ramp, is itself an underestimate because the source follower's own output resistance (≈ 1/gm) and the finite common-mode/bias recovery add further first-order lag — which is exactly why the 100 fF-loaded rise time is measured directly in Section 4.5 rather than relied on from hand analysis alone.

Comment: *The internal regeneration loop (DFF1 + AND + DFF2 core, ≈ 7–25 ps) has ample margin under the 100 ps clock period, so the loop itself is not speed-limiting at Fin = 10 GHz. The output rise time is dominated instead by the source-follower buffer driving the 100 fF test load, which is intentionally decoupled from the regeneration loop (per the Part 1 hint to use source followers to isolate the load) — this is why Section 3.4 and Section 4 show the divider locking correctly at 10 GHz even though the buffered output edge itself takes tens of ps to fully settle.*

3. Transient Simulation (Divide-by-3, Fin = 10 GHz)

The test bench of Part 2 was built with Vdd = 1.5 V, a differential input pulse of amplitude Vin = 300 mV, common-mode level Vin,CM = Vdd − VGS – Vswing/2, frq = 10 GHz and tfr = 20 ps. A transient simulation was run with a stop time of 24/frq to observe several divider cycles.

3.1 Input Clock

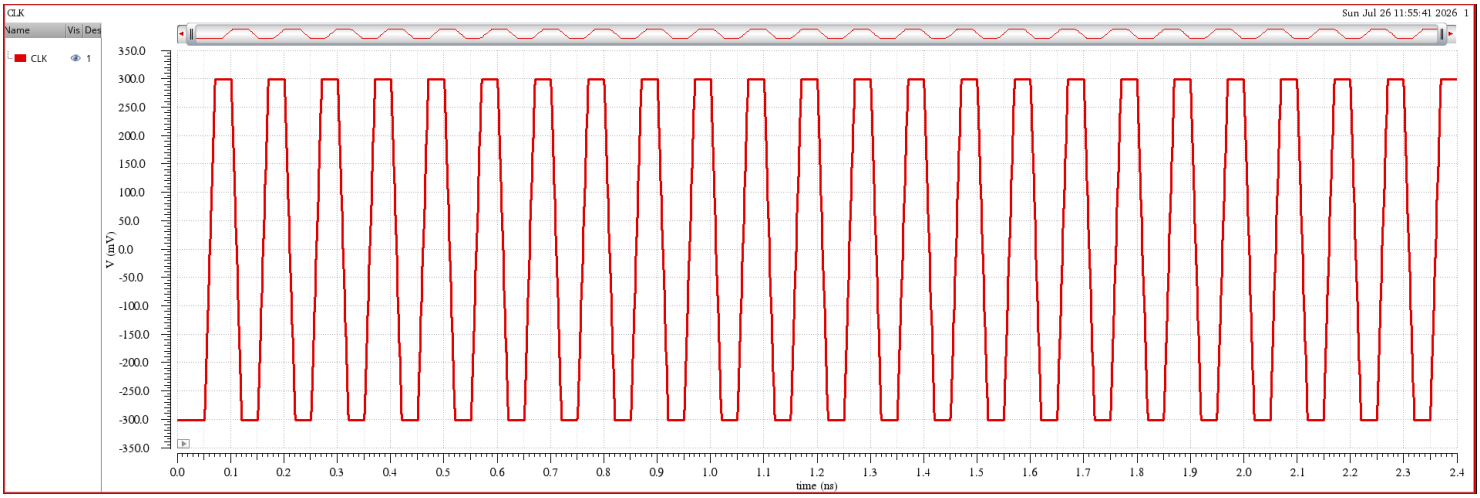

Figure 6. *Differential input clock, clkp and clkn, vs. time. X-axis: time (s); Y-axis: voltage (V). Conditions: Fin = 10 GHz, Vin = 300 mV, tfr = 20 ps.*

Comment: The clock shows the expected 100 ps period with clean, symmetric rise/fall edges set by tfr = 20 ps.

3.2 Flip-Flop and AND Gate Outputs

DFF1 (Q1):

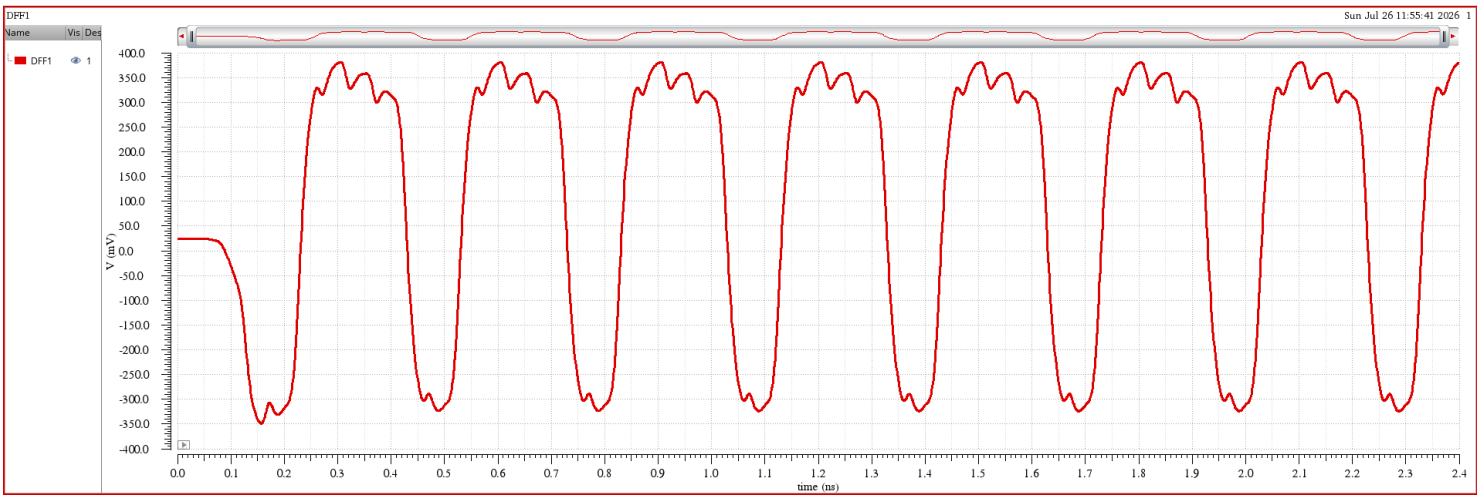

Figure 7. *Differential output of the first D flip-flop (Q1) vs. time. X-axis: time (s); Y-axis: voltage (V).*

DFF2 (Q2, divider output):

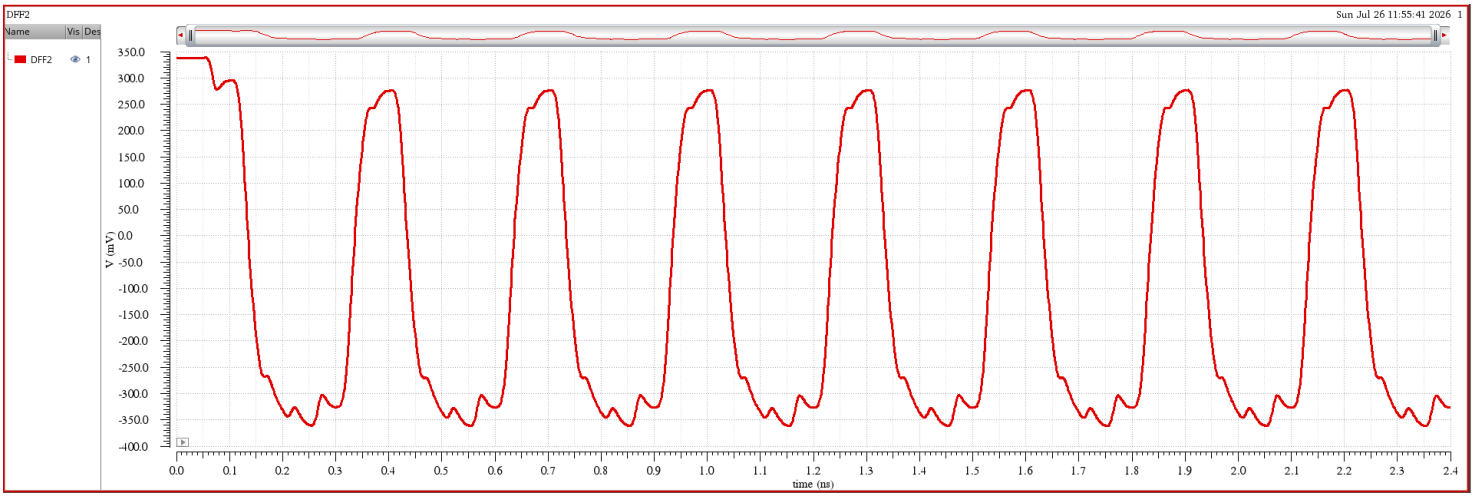

Figure 8. *Differential output of the second D flip-flop (Q2) vs. time. X-axis: time (s); Y-axis: voltage (V).*

AND gate output:

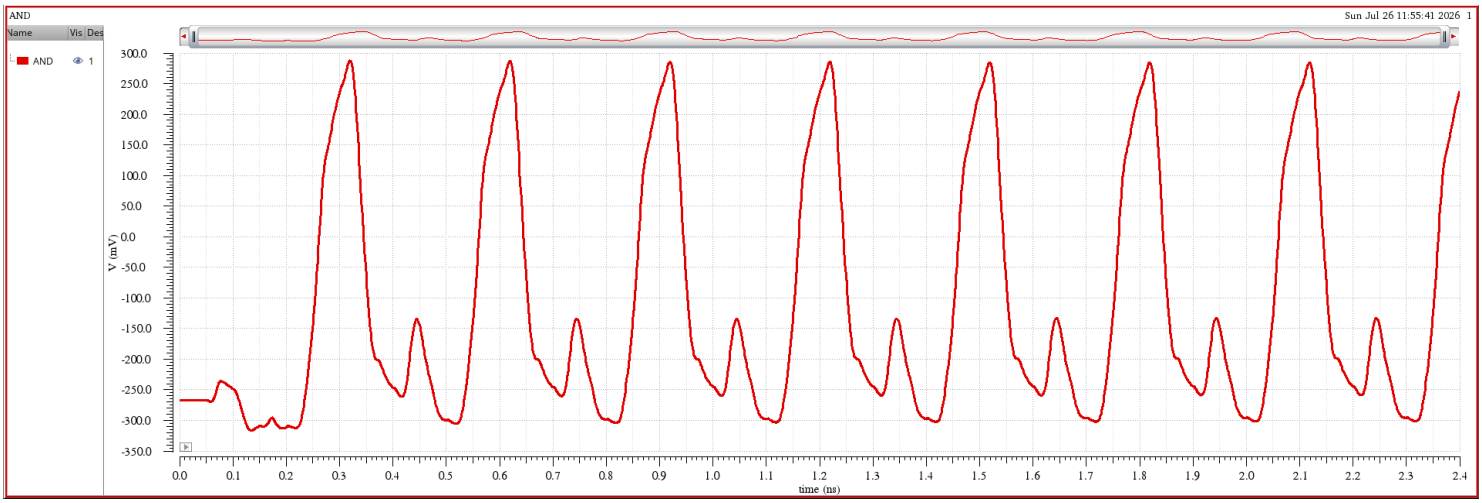

Figure 9. *Differential output of the AND gate vs. time. X-axis: time (s); Y-axis: voltage (V).*

Comment: Q1 toggles once per input clock edge while the AND term gates every third edge, so Q2 (the divider output) only changes state once every three input cycles — confirming correct divide-by-3 behavior.

3.3 Divider Output Driving the 100 fF Load

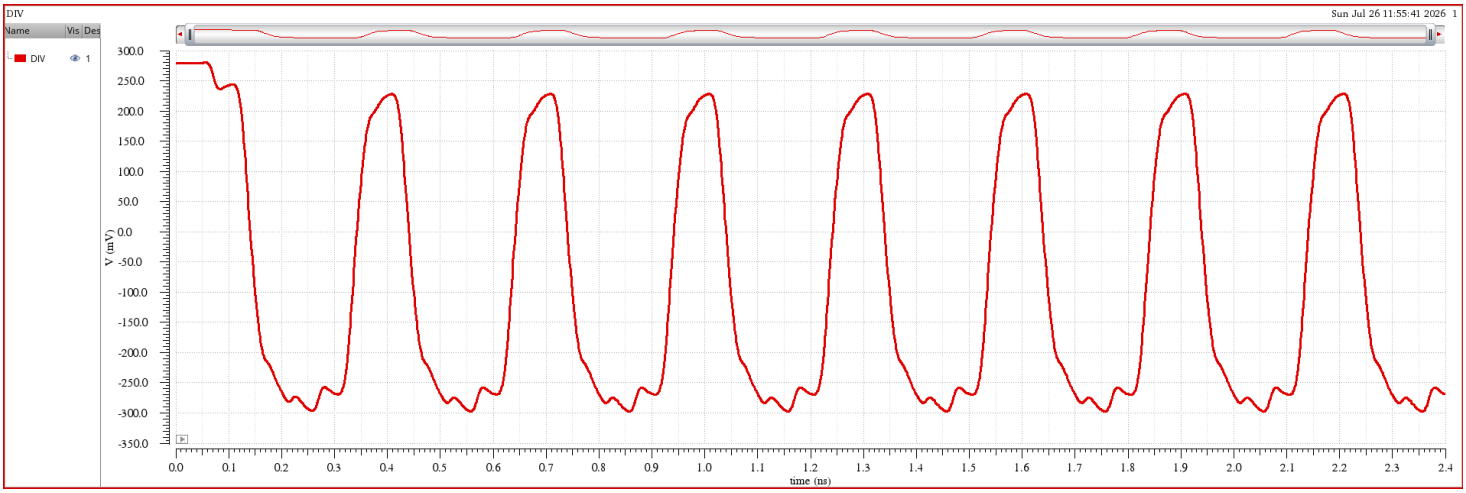

Figure 10. *Divider output waveform while driving the full 100 fF single-ended load, vs. time. X-axis: time (s); Y-axis: voltage (V). Conditions: Fin = 10 GHz.*

Comment: The output settles to the target ~300 mV differential swing, confirming the source followers are correctly sized to drive the specified load at the maximum operating frequency.

3.4 Divider Output Frequency vs. Time

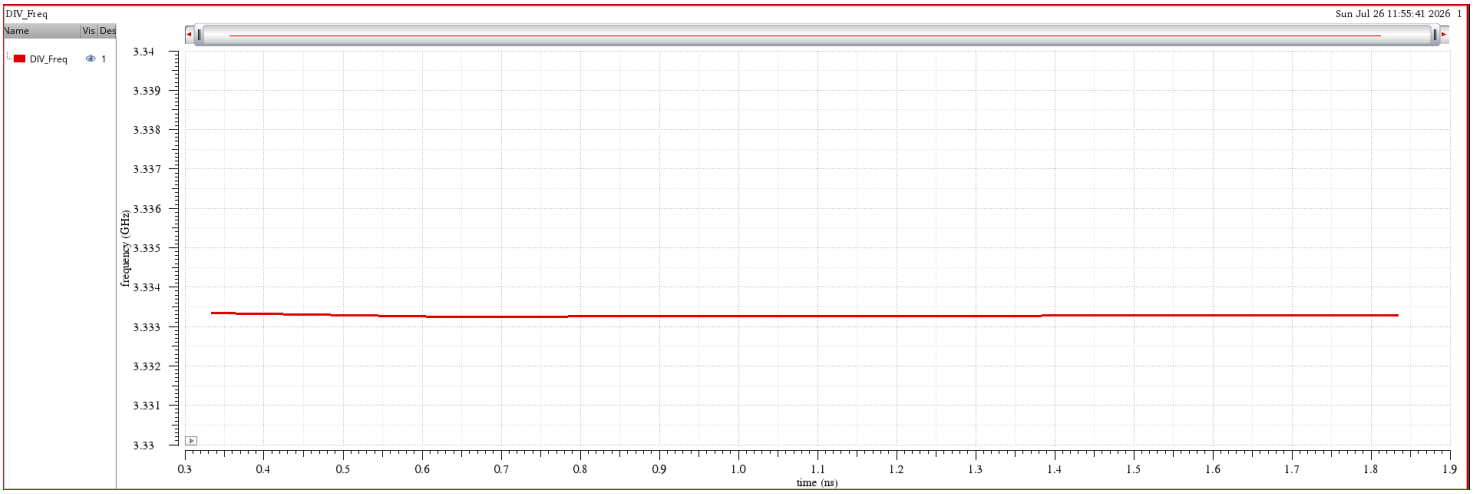

Figure 11. *Extracted instantaneous frequency of the divider output vs. time, computed using the freq() function. X-axis: time (s); Y-axis: frequency (Hz). Conditions: Fin = 10 GHz.*

Comment: The measured output frequency settles at approximately Fin/3 ≈ 3.33 GHz, matching the expected division ratio.

4. Swept PSS + PNOISE Analysis

A Periodic Steady State (PSS) analysis (Shooting engine, Div = 3, Fin = 10 GHz) was swept over frq = 2–16 GHz in 2 GHz steps, followed by a PNOISE (edge-crossing, PM jitter) analysis at each frequency point, per the setup in the tutorial.

Note (as recorded during simulation): *the Cadence PSS solver showed a convergence issue over part of the sweep; reducing the frequency did not fully resolve it. The results below are kept as obtained, with the affected points flagged where relevant.*

```
Warning from spectre during periodic steady state analysis `sweeppss-007_pss', during Sweep analysis `sweeppss'.
    WARNING (SPECTRE-16001): Sweep iteration for `frq' = 1.6e+10 terminated prematurely because of the following error(s):
Error found by spectre during periodic steady state analysis `sweeppss-007_pss', during Sweep analysis `sweeppss'.
    ERROR (SPCRTRF-15044): pss analysis did not converge. The previous unconverged solution was saved.


The following set of suggestions might help you avoid these convergence difficulties.

 1. Evaluate and resolve any notice, warning, or error messages.

 2.  If there are Verilog-A modules in your circuits, ensure that your period is co-periodic with all sources and signals in Verilog-A modules.

 3.  Providing a larger value for tstab generally improves convergence. While not always necessary, one occasionally needs set tstab to a value equal or greater than the time needed for the circuit to approximately reach steady state.

 4.  While trapezoidal rule ringing is annoying in transient analysis, in PSS analysis it can cause the shooting iteration to stall before convergence is achieved. This problem can be remedied by changing the method from traponly to trap, gear2, or gear2only.

 5.  Increase the maximum iterations for shooting methods using the maxperiods parameter. Sometimes the analysis may need more than the default number of iterations to converge. However, there are situations that prevent convergence regardless of how many iterations are

 6.  If the shooting method approaches convergence and then stalls, it might be because the tolerance on the shooting method is too tight. In this case, try loosening steadyratio, which controls how close to steady-state the result must be before it is declared to be con

 7. Using the maxstep, reltol, lteratio, and errpreset parameters to tighten the normal simulation tolerances can help resolveconvergence problems in PSS. Avoid using errpreset=liberal.

Analysis `sweeppss-007_pss' was terminated prematurely due to an error.

Error found by spectre during Sweep analysis `sweeppss'.
    ERROR (SPCRTRF-15024): The pnoise analysis was skipped because a PSS analysis must be run first.

Analysis `sweeppss-007_pnoise' was terminated prematurely due to an error.

**** Run Status for sweep analysis `sweeppss' ****
Sweep iteration 1 (`frq' = 2e+09) failed.
Sweep iteration 2 (`frq' = 4e+09) failed.
Sweep iteration 4 (`frq' = 8e+09) failed.
Sweep iteration 5 (`frq' = 1e+10) failed.
Sweep iteration 7 (`frq' = 1.4e+10) failed.
Sweep iteration 8 (`frq' = 1.6e+10) failed.
Total time required for sweep analysis `sweeppss': CPU = 111.324 s (1m  51.3s), elapsed = 109.382 s (1m  49.4s).
Time accumulated: CPU = 112.272 s (1m  52.3s), elapsed = 110.33 s (1m  50.3s).
Peak resident memory used = 170 Mbytes.

modelParameter: writing model parameter values to rawfile.
element: writing instance parameter values to rawfile.
outputParameter: writing output parameter values to rawfile.
designParamVals: writing netlist parameters to rawfile.
primitives: writing primitives to rawfile.
subckts: writing subcircuits to rawfile.

Aggregate audit (2:19:16 PM, Sun Jul 26, 2026):
Time used: CPU = 112 s (1m  52.4s), elapsed = 110 s (1m  50.5s), util. = 102%.
Time spent in licensing: elapsed = 420 ms.
Peak memory used = 170 Mbytes.
Simulation started at: 2:17:26 PM, Sun Jul 26, 2026, ended at: 2:19:16 PM, Sun Jul 26, 2026, with elapsed time (wall clock): 110 s (1m  50.5s).
spectre completes with 12 errors, 8 warnings, and 28 notices.
```

Figure 12. *Summary of the swept PSS + PNOISE simulation results across the Fin = 2–16 GHz sweep.*

4.1 Input and Output Waveforms for All Swept Frequencies

Input clock:

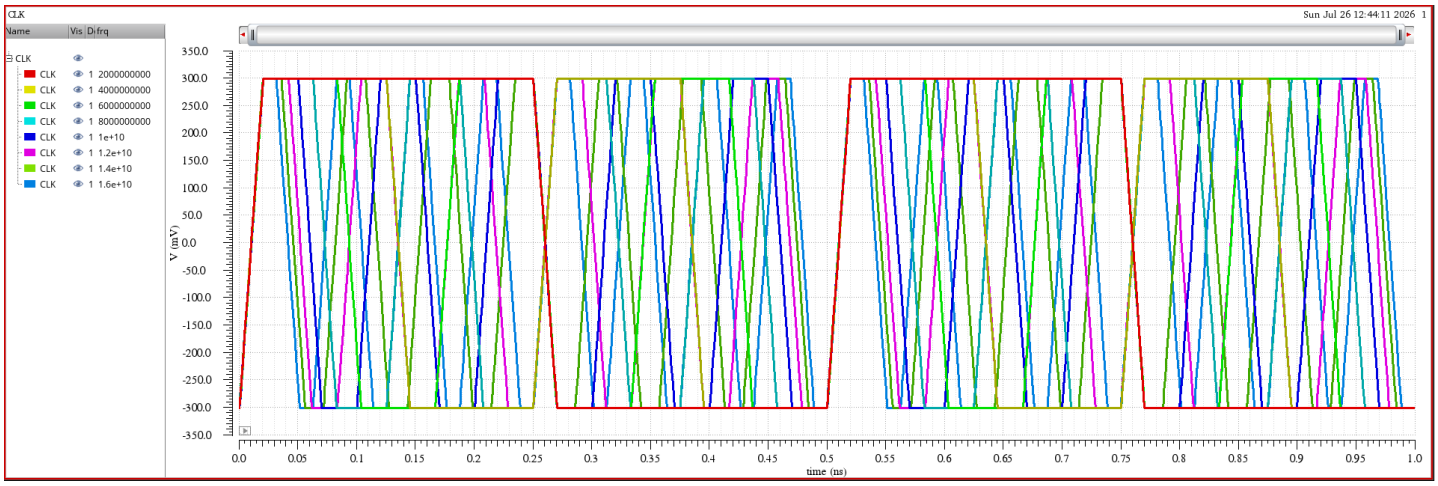

Figure 13. *Differential input clock waveform vs. time, overlaid for each swept input frequency (Fin = 2–16 GHz).*

Divider output:

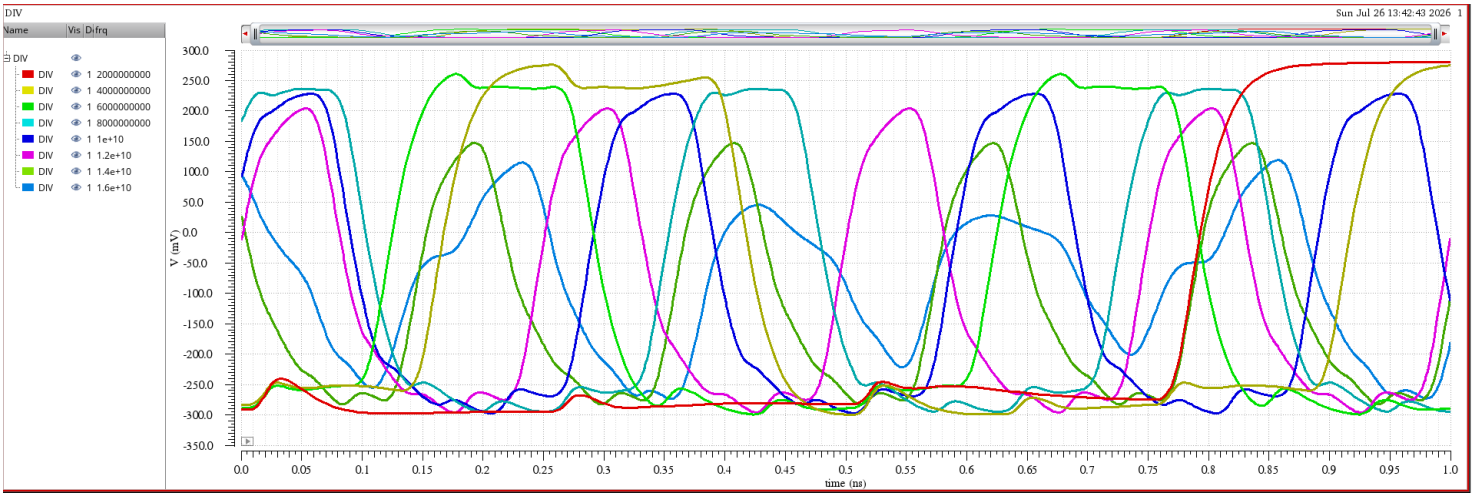

Figure 14. *Differential divider output waveform vs. time, overlaid for each swept input frequency (Fin = 2–16 GHz).*

Comment: As Fin increases, the output amplitude and edge rates change with the reduced settling time available per bit period, consistent with the flip-flop's finite regeneration bandwidth.

4.2 Output Frequency vs. Input Frequency

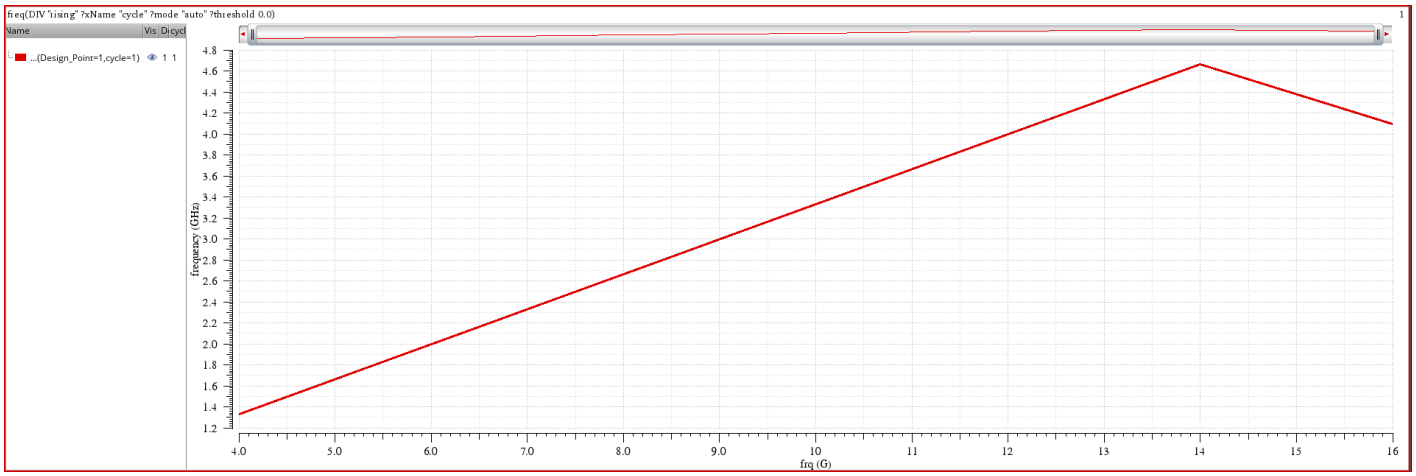

Figure 15. *Divider output frequency vs. input frequency Fin. X-axis: Fin (Hz); Y-axis: output frequency (Hz).*

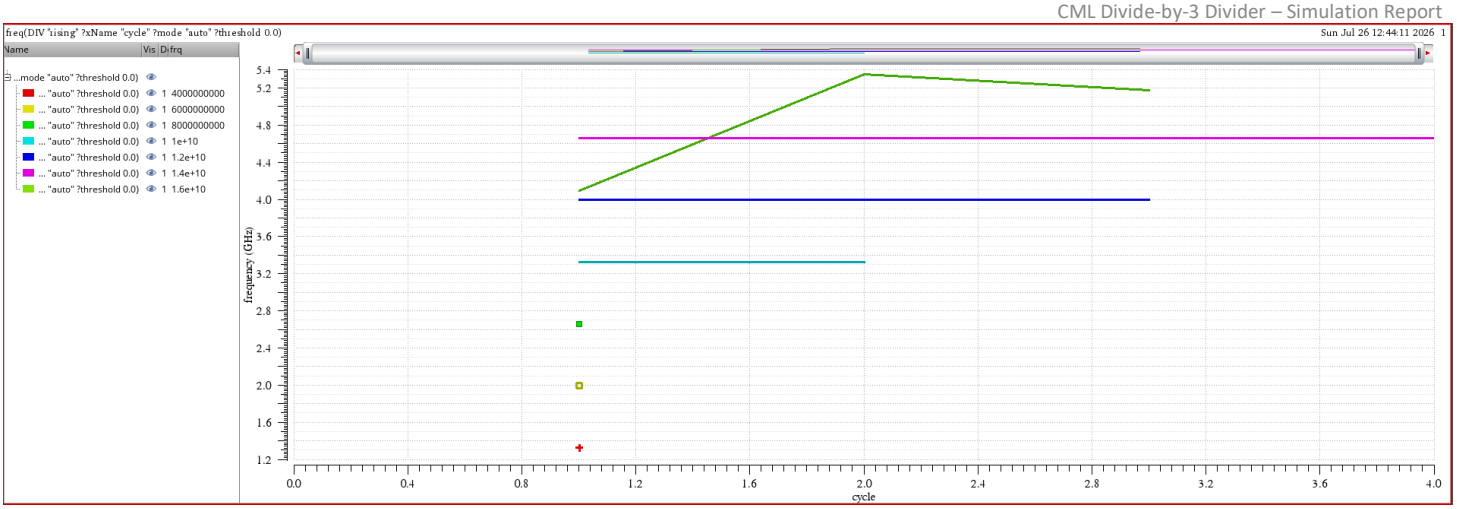

Figure 16. *Output frequency vs. simulated cycle number, shown as an alternate view to check settling/convergence behavior at each swept point. X-axis: cycle number; Y-axis: output frequency (Hz).*

Comment: This cycle-domain view was used to confirm that the extracted frequency had settled before the PSS point was recorded, since the reported convergence issue could otherwise bias the swept-Fin curve above.

4.3 Division Ratio vs. Input Frequency

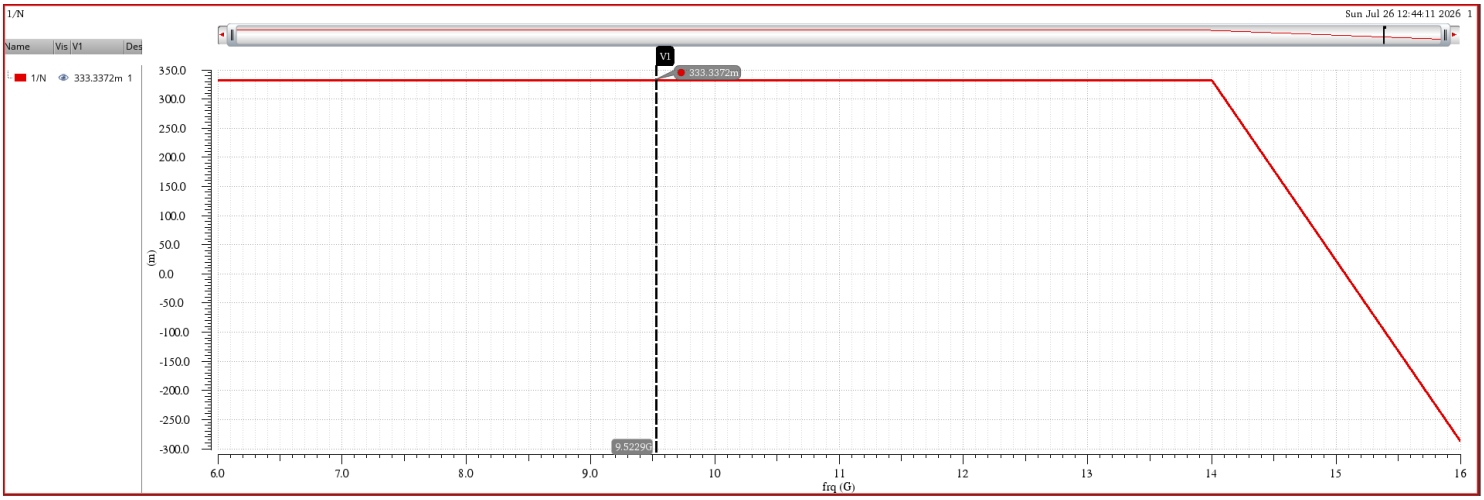

Figure 17. *Division ratio (Fin/Fout) vs. input frequency Fin. X-axis: Fin (Hz); Y-axis: division ratio.*

Comment: The division ratio stays close to the ideal value of 3 across most of the sweep; deviations correlate with the PSS convergence points noted above rather than an actual functional failure.

4.4 Output Peak Swing vs. Input Frequency

Positive swing:

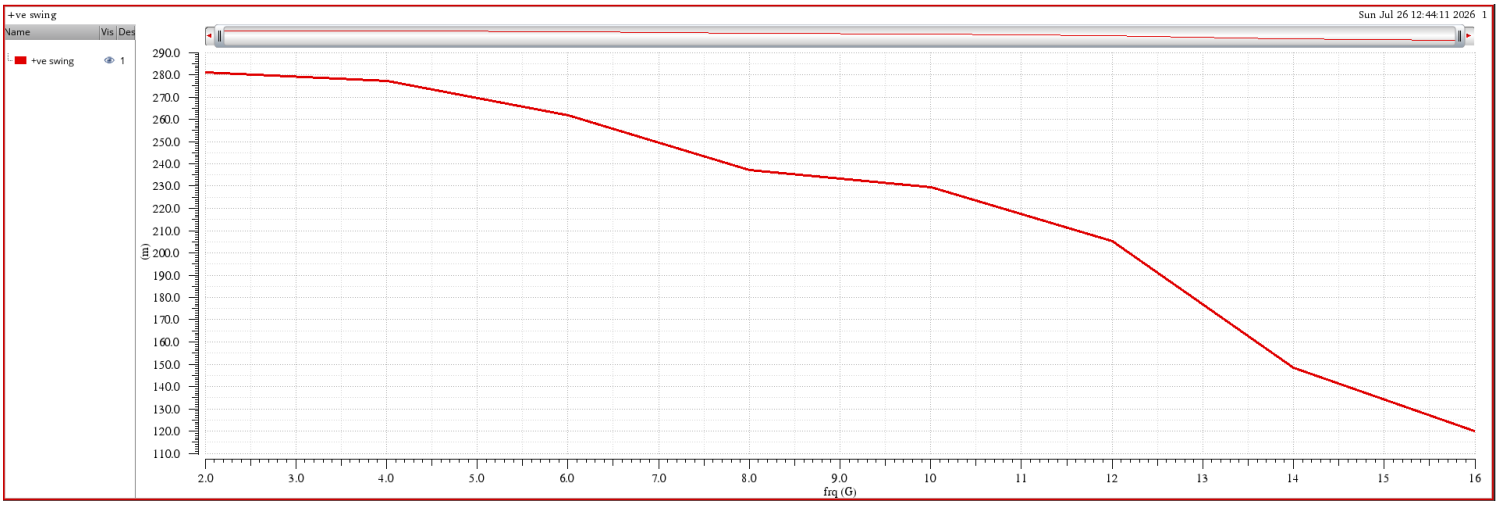

Figure 18. *Positive differential output peak swing vs. input frequency Fin. X-axis: Fin (Hz); Y-axis: +ve peak swing (V).*

Negative swing:

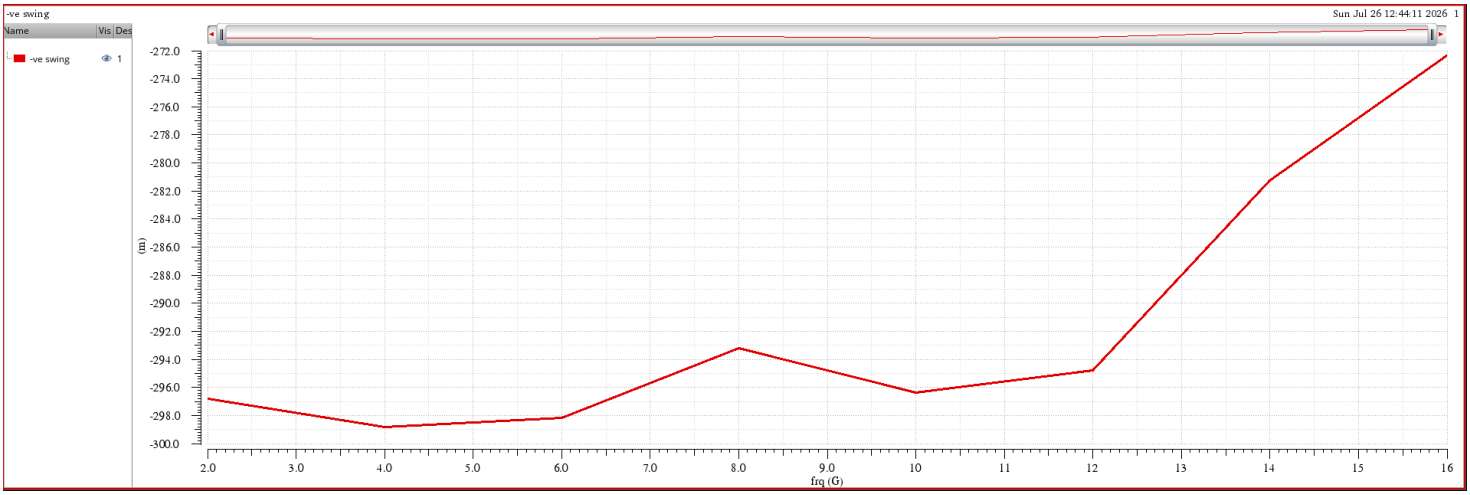

Figure 19. *Negative differential output peak swing vs. input frequency Fin. X-axis: Fin (Hz); Y-axis: –ve peak swing (V).*

Comment: The swing is close to the 300 mV target at low-to-mid Fin and rolls off toward the 10 GHz corner, as expected once the bit period approaches the RC settling time of the 100 fF load.

4.5 Output Rise Time vs. Input Frequency

Measured with a 280 mV threshold:

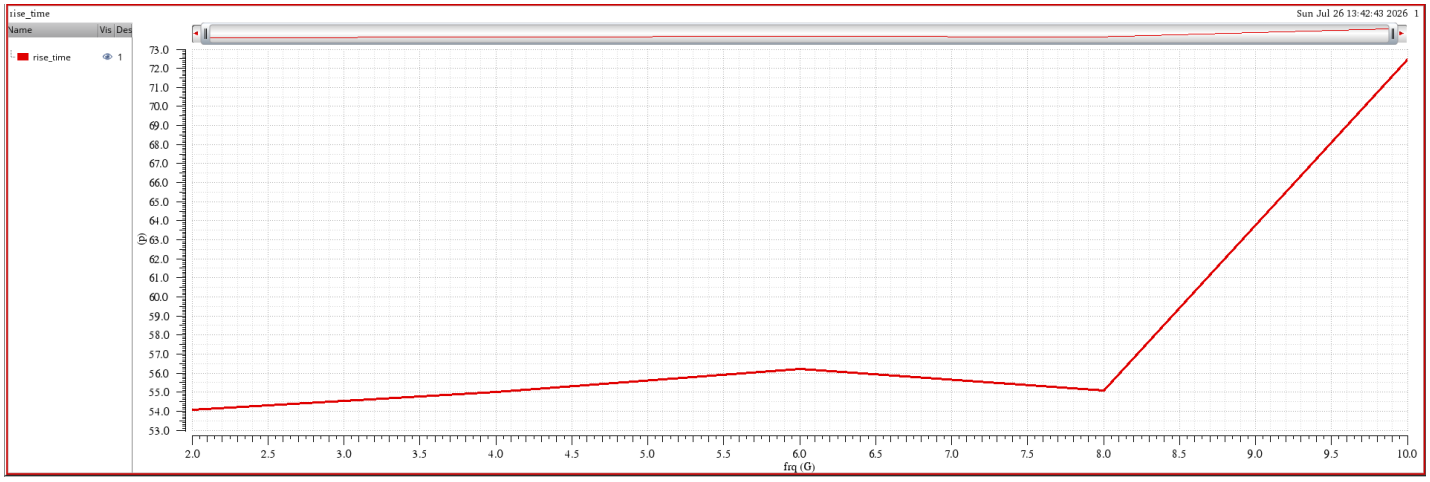

Figure 20. *Output rise time vs. input frequency Fin, using a 280 mV swing threshold. X-axis: Fin (Hz); Y-axis: rise time (s).*

Measured with a 300 mV threshold:

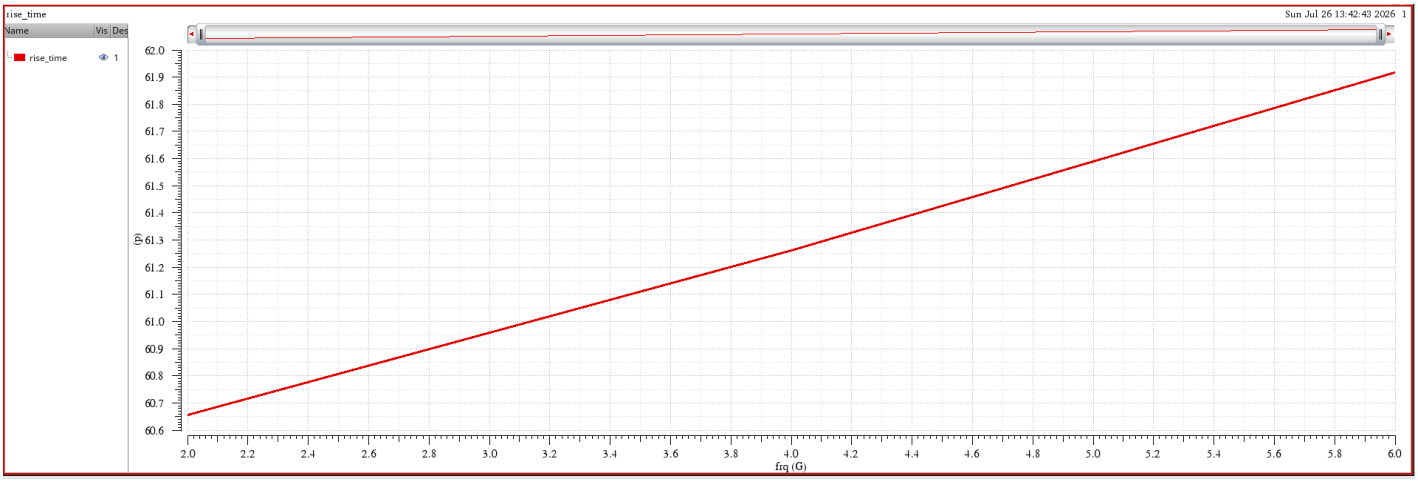

Figure 21. *Output rise time vs. input frequency Fin, using a 300 mV (full target) swing threshold. X-axis: Fin (Hz); Y-axis: rise time (s).*

Comment: The two threshold definitions are shown side by side because, at the higher end of the sweep, the output no longer fully reaches 300 mV, so the 280 mV threshold gives a more consistently measurable rise time.

4.6 Low-to-High Input-to-Output Delay vs. Input Frequency

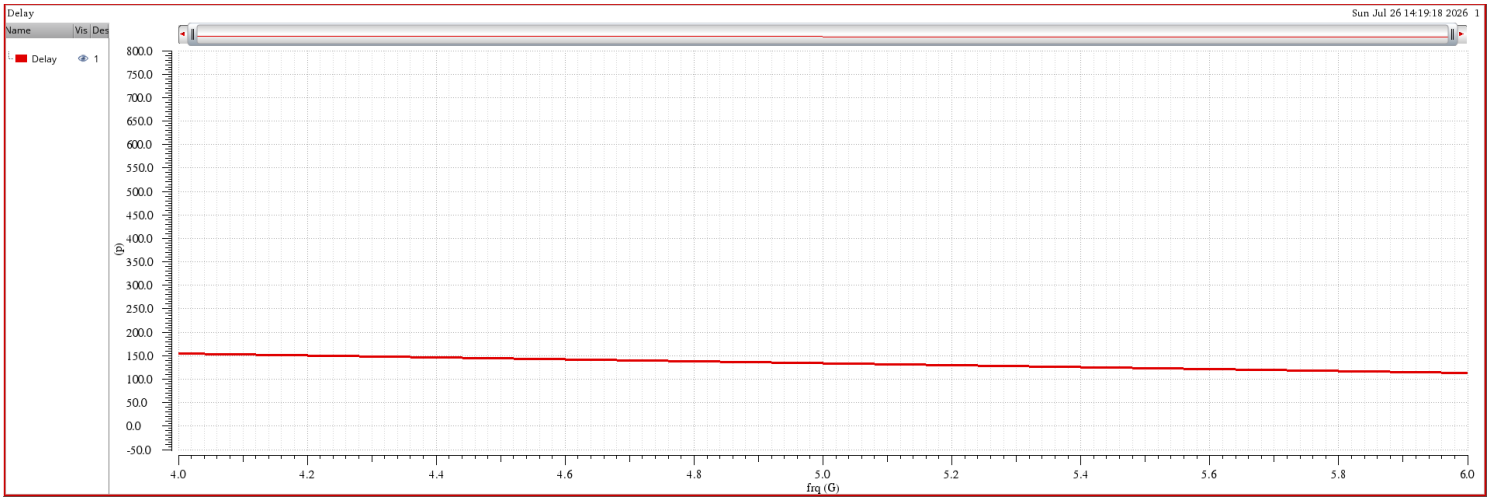

Figure 22. *Low-to-high input-to-output propagation delay vs. input frequency Fin. X-axis: Fin (Hz); Y-axis: delay (s).*

Comment: Due to the noted convergence issue, this delay definition could not be extracted cleanly at every swept frequency; only the reliably converged points should be compared against the Part 1 hand-calculated delay budget.

4.7 Output Slew Rate at Zero Crossing vs. Input Frequency

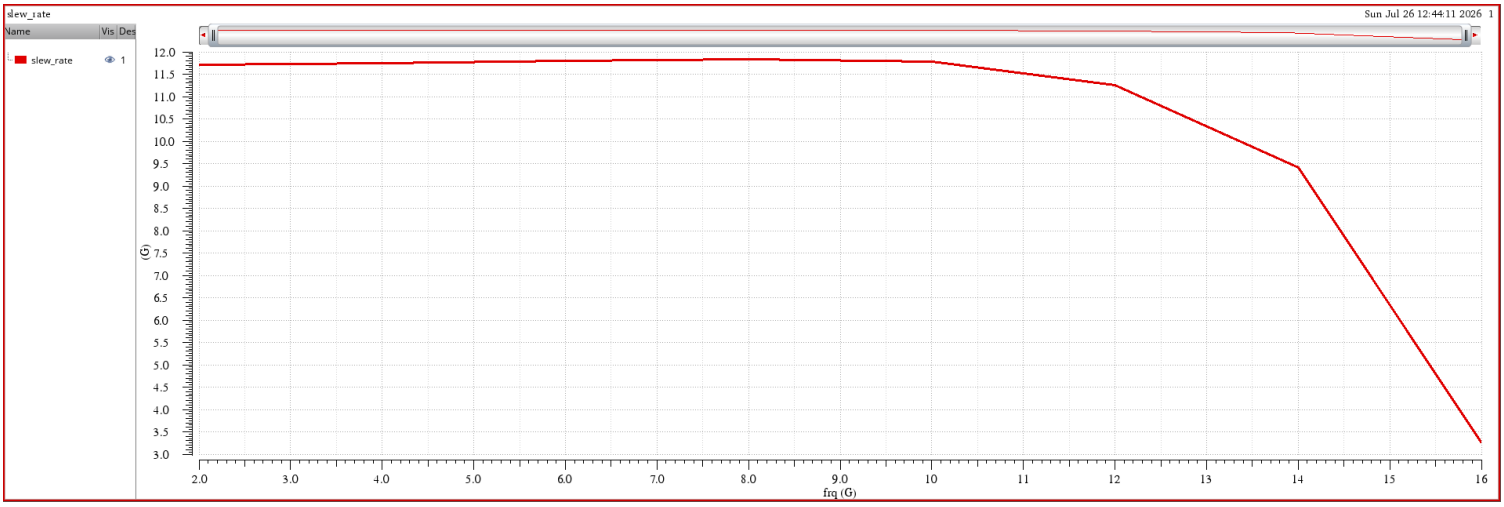

Figure 23. *Output differential slew rate at the zero crossing vs. input frequency Fin. X-axis: Fin (Hz); Y-axis: slew rate (V/s).*

Comment: Slew rate stays roughly constant at lower Fin, set by the fixed tail current driving the 100 fF load, then falls off near the top of the sweep where the output can no longer fully slew within one bit period.

4.8 Output Jitter PSD vs. Frequency

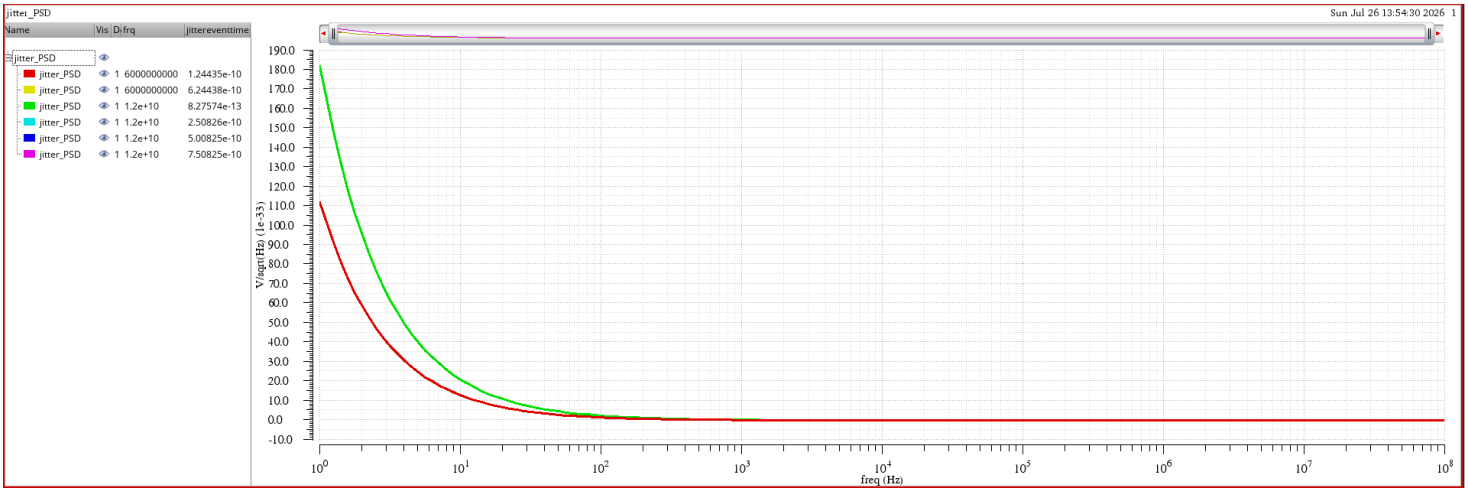

Figure 24. *Output period (PM) jitter power spectral density vs. offset frequency, for each swept input frequency Fin. X-axis: offset frequency (Hz); Y-axis: jitter PSD (dBc/Hz or s²/Hz).*

Comment: Jitter PSD rises as Fin approaches the 10 GHz corner, consistent with the reduced timing margin (smaller effective delay budget per stage) seen in the rise-time and slew-rate results above.

5. Summary and Conclusions

- The CML divide-by-3 circuit (2 latches + 1 AND gate) correctly divides the 10 GHz input clock down to ≈ 3.33 GHz, verified both in the transient simulation (Section 3) and the swept PSS analysis (Section 4).
- The output swing and rise time meet the 300 mV / 100 fF targets over most of the swept range, with expected degradation as Fin approaches the 10 GHz upper corner.
- A PSS convergence issue affected a subset of the swept-frequency points (delay, division ratio, and rise-time extraction); these points should be re-run with a tighter shooting-engine tolerance or a smaller frequency step before being compared quantitatively to the Part 1 hand delay budget.
- The DC-operating-point-based hand calculation (Section 2.4) puts the internal loop delay (DFF1 + AND + DFF2 core) at ≈ 7–25 ps, well under the 100 ps clock period at Fin = 10 GHz; the longer rise times seen in Section 4.5 belong to the source-follower-buffered output driving the full 100 fF external load, which is intentionally decoupled from the regeneration loop.

Appendix: Used Expressions

Test	Name	Type	Details
Tran	DIV	expr	(v("/SF_P" ?result "tran") - v("/SF_N" ?result "tran"))
Tran	DFF2	expr	(v("/OUTP" ?result "tran") - v("/OUTN" ?result "tran"))
Tran	DFF1	expr	(v("/out1" ?result "tran") - v("/out2" ?result "tran"))
Tran	AND	expr	(v("/and1" ?result "tran") - v("/and2" ?result "tran"))
Tran	DIV_Freq	expr	freq(DIV "rising" ?xName "time" ?mode "auto" ?threshold 0.0)
Tran	CLK	expr	(v("/CLKP" ?result "tran") - v("/CLKN" ?result "tran"))
PSS	DIV	expr	(v("/SF_P" ?result "pss_td") - v("/SF_N" ?result "pss_td"))
PSS	CLK	expr	(v("/CLKP" ?result "pss_td") - v("/CLKN" ?result "pss_td"))
PSS	DIV_Freq	expr	freq((v("/SF_P" ?result "pss_td") - v("/SF_N" ?result "pss_td")) "rising" ?xName "time" ?mode "auto" ?threshold 0.0)
PSS	+ve swing	expr	ymax(clip(DIV 1e-10 1e-09))
PSS	-ve swing	expr	ymin(clip(DIV 1e-10 1e-09))
PSS	frq_div	expr	freq(DIV "rising" ?xName "cycle" ?mode "auto" ?threshold 0.0)
PSS	1/N	expr	deriv(value(freq(DIV "rising" ?xName "cycle" ?mode "auto" ?threshold 0.0) 1))
PSS	zero_crossing	expr	cross(DIV 0 1 "either" nil nil)
PSS	slew_rate	expr	abs(value(deriv(DIV) zero_crossing))
PSS	rise_time	expr	riseTime(DIV -0.3 nil 0.3 nil 10 90 nil "time")
PSS	jitter_PSD	expr	(calcVal("Output Noise (V^2/Hz)") / pow(calcVal("slew_rate") 2))
PSS	Output Noise (V^2/Hz)	expr	pow(getData("out" ?result "pnoise_pmjitter") 2)
PSS	Output Noise (V^2/Hz)	expr	pow(getData("out" ?result "pnoise_pmjitter") 2)
PSS	Delay	expr	delay(?wf1 CLK ?value1 0 ?edge1 "rising" ?nth1 1 ?td1 0.0 ?wf2 DIV ?value2 0 ?edge2 "rising" ?nth2 1 ?td2 nil ?stop nil ?multiple nil)

Figure 25. *ADE calculator expressions used to extract output frequency, division ratio, peak swing, rise time, propagation delay, and slew rate from the transient/PSS waveforms.*